WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Créé par le Centre d'Excellence Cyberdéfense Aérospatiale (CEC)

Guide des tests
WAC (Windows Artefacts Collector)

Logiciel de prélèvement d'artefacts Windows pour l'investigation numérique



ARTEFACTS SYSTÈME

FICHIERS UTILISATEUR

PREUVES NUMÉRIQUES

UN OUTIL AU SERVICE DE L'INVESTIGATION NUMÉRIQUE

WAC — Windows Artefacts Collector

Guide des tests

Version du logiciel	2.0.0
Édition du document	septembre 2026
Éditeur	Centre d'Excellence Cyberdéfense Aérospatiale (CEC)
	École de l'air et de l'espace, base aérienne 701, Salon-de-Provence

Ce document explique comment lancer à la main chacun des tests de WAC, ce que chacun prouve et ne prouve pas, et comment lire un échec. Il s'adresse à qui modifie le code ou veut vérifier une version avant de l'employer. Les durées et volumes cités ont été relevés sur la machine virtuelle de test et sur une collecte réelle ; ce sont des ordres de grandeur. Le bilan de ces tests, et ce qu'ils garantissent, est dans le *Dossier de validation*.

Table des matières

1 Principes **3**

1.1 Ce qu'un test de WAC doit prouver . . . 3

1.2 Trois règles . . . 3

2 Préparer l'environnement **4**

2.1 Le poste Linux . . . 4

2.2 Construire WAC et les harnais . . . 4

2.3 La machine virtuelle de test . . . 5

3 Le cycle de test dans la machine virtuelle **6**

3.1 Lancer un cycle . . . 6

3.2 Contrôler les sorties . . . 7

3.3 Comparer deux passages . . . 9

4 Les harnais de format **10**

4.1 Empreintes : `sha_test` . . . 10

4.2 Lecture JSON : `json_test` . . . 10

4.3 Décompression : `lznt1_test` et `xpress_test` . . . 10

4.4 Artefacts par version de Windows : `artefact_catalogue_test` . . . 11

4.5 Sessions des navigateurs : `browser_sessions_test` . . . 12

4.6 Décompression LZX : `lzx_test` . . . 12

4.7 Messages d'événements : `wevt_test` et `event_messages_test` . . . 12

4.8 Journaux : `evtx_test` . . . 13

4.9 Lecteur de ruches : `offline_registry_test` . . . 13

4.10 Conversions : `system_conversions_test` . . . 14

4.11 Consigne : `consigne_test` . . . 14

4.12 Rejeu des journaux de ruche : `hive_recover_test` . . . 14

4.13 Clé de hachage de la SAM : `sam_secrets_test` . . . 15

4.14 Scripts de vérification : `verify_exhibits_test.sh` . . . 15

4.15 Lecture brute : `raw_hive_test` . . . 16

4.16 Enregistrements NTFS : `ntfs_fixup_test` . . . 16

4.17 `$MFT` et journal des modifications : `mft_test`, `usn_journal_test` 16
4.18 Lecture brute sur un vrai volume : `raw_volume_test.sh` 17
4.19 Courriels d'Outlook : `pst_test` 17
4.20 Index d'Internet Explorer : `ie_index_dat_test` 17
4.21 Chronologie Windows : `activities_cache_test` 18
4.22 Petits artefacts fichiers : `file_artefacts_test` 18
4.23 Signatures : `authenticode_test` 19
4.24 Configuration : `yaml_light_test`, `config_test` 19
4.25 Chaînes tierces : `third_party_chain_test` 20
4.26 Pilotes vulnérables : `version_info_test`, `vulnerable_drivers_test` 20
4.27 Listes de confiance : `inflate_test`, `cab_test`, `zip_test`, `csv_reader_test`, `trust_list_test` 21

5 Les harnais de robustesse **23**
5.1 Le principe 23
5.2 Raccourcis : `lnk_test` 23
5.3 Listes de sauts et Prefetch : `parsers_test` 23
5.4 Référence 24

6 Recouper sur une collecte réelle **25**

7 Lire un échec **26**
7.1 Codes de sortie 26
7.2 Localiser une lecture hors tampon 26
7.3 Une boucle sans fin 27

CHAPITRE 1

Principes

1.1 Ce qu'un test de WAC doit prouver

WAC produit des fichiers JSON. Qu'ils soient valides ne prouve presque rien : un champ lu au mauvais décalage, une date interprétée dans le mauvais fuseau, un nom décodé dans la mauvaise page de code donnent un JSON parfaitement valide — et faux. Les défauts les plus coûteux relevés dans WAC étaient tous de cette sorte : ils ne se voyaient qu'en confrontant une valeur à une autre source.

Les tests se rangent donc en trois familles, qui ne répondent pas à la même question :

Le cycle dans la machine virtuelle (chapitre 3) : WAC s'exécute-t-il jusqu'au bout sur un vrai Windows, et ses sorties sont-elles cohérentes entre elles ? Les *contrôles croisés* confrontent une valeur calculée par WAC à une valeur indépendante de la même collecte.

Les harnais de format (chapitre 4) : un décodeur donne-t-il exactement le résultat de la référence — l'outil de Microsoft, une norme publiée, une autre implémentation ?

Les harnais de robustesse (chapitre 5) : un analyseur lit-il en dehors de son tampon, ou boucle-t-il sans fin, sur un fichier tronqué ou forgé ?

1.2 Trois règles

1. **Un correctif s'accompagne du test qui aurait pris le défaut.** Et ce test se vérifie dans les deux sens : il passe sur la sortie corrigée, et il échoue quand on rejoue le défaut.
2. **Après tout changement, comparer au passage précédent**, et pas seulement lire les pastilles vertes : un artefact passé de 1 522 entrées à 573 n'allume aucun voyant (section 3.3).
3. **Une donnée réelle ne quitte pas le poste de test.** Une collecte réelle contient des données personnelles : elle se traite en local, ses copies temporaires s'effacent après usage, et rien n'en entre dans le dépôt (`vmtest/results/` est ignoré par git).

CHAPITRE 2

Préparer l'environnement

2.1 Le poste Linux

Tout se construit sur Linux, par compilation croisée MinGW-w64 ; les harnais tournent soit nativement, soit sous `wine`, soit dans la machine virtuelle.

```
sudo apt install g++-mingw-w64-x86-64 wine64 python3 python3-pip
pip install olefile          # lecteur OLE indépendant, pour les recoupements
pip install dissect.esedb    # lecteur ESE indépendant : WebCache (sinon contrôle sauté)
# ou, sans toucher au Python du système : vmtest/tools/ese-venv
python3 -m venv vmtest/tools/ese-venv && vmtest/tools/ese-venv/bin/pip install dissect.
    esedb==3.18
```

2.2 Construire WAC et les harnais

```
./build-windows.sh --test
```

Produit, dans `build-windows/`	**Rôle**
`WAC.exe`	l'outil de collecte
`raw_hive_test.exe`	lecteur NTFS brut, utilisé par le cycle VM
`tests/lnk_test.exe`	robustesse de l'analyseur de raccourcis
`tests/parsers_test.exe`	robustesse des listes de sauts et des Prefetch
`tests/evtx_test.exe`, `consigne_test.exe`	décodeur de journaux, consigne (chapitre 4)
`tests/offline_registry_test.exe`	lecteur de ruches confronté à celui de Microsoft
`tests/system_conversions_test.exe`	conversions de WAC confrontées à celles de Windows
`tests/ntfs_fixup_test.exe`	vérification des enregistrements NTFS (enregistrements déchirés)
`tests/mft_test.exe`, `usn_journal_test.exe`	`$MFT` et journal des modifications confrontés à ntfs-3g et aux structures du compilateur

> **Attention**
> Les harnais sont volontairement dans `build-windows/tests/`, à l'écart de `WAC.exe` : ce sont des outils de test, qui n'ont rien à faire sur un support de collecte. Ne jamais y copier le dossier `tests/`.

La compilation doit se terminer sans aucun avertissement : les options `-Wall -Wextra` sont actives, et un avertissement a déjà signalé de vrais défauts.

Les autres harnais (chapitre 4) se construisent chacun par une commande donnée dans son en-tête et reprise dans ce guide ; plusieurs compilent nativement sous Linux.

2.3 La machine virtuelle de test

La VM est un Windows 11 piloté par le *qemu-guest-agent* : aucune interaction, aucun réseau, les commandes s'exécutent en `NT AUTHORITY\SYSTEM`. Sa création et ses prérequis sont décrits dans `vmtest/README_FR.md` ; en résumé :

```
cd vmtest
ISO_WIN=~/Téléchargements/Win11_25H2_French_x64_v2.iso ./create-vm.sh   # une fois
./install-browsers.sh                          # Chrome et Firefox, pour web_client (une
    fois)
python3 lib/qga.py ping                        # l'agent répond ?
python3 lib/qga.py run --shell "dir C:\\"      # exécuter une commande
python3 lib/qga.py write <hôte> <invité>       # envoyer un fichier
python3 lib/qga.py read  <invité> <hôte>       # rapatrier un fichier
```

> **À retenir**
> Une commande lancée par `qga.py run` est coupée au bout de 30 minutes. Pour un test plus long (par exemple les Prefetch d'une machine réelle, section 5.3), le lancer en tâche détachée dans la VM et suivre son journal.

CHAPITRE 3

Le cycle de test dans la machine virtuelle

3.1 Lancer un cycle

```
cd vmtest
./run-wac-test.sh --build     # recompile, puis cycle complet
./run-wac-test.sh             # cycle complet avec les binaires déjà construits
./run-wac-test.sh --raw-only  # seulement la validation de la lecture brute
./run-wac-test.sh --no-split  # sans l'étape 7 (collecte puis conversion)
./run-wac-test.sh --binary-all  # étape 7 avec --binary-all
./run-wac-test.sh --only=mft,usn_journal  # seulement ces artefacts
```

`--only` prend des clés de la section `artefacts:` de `wac.yml` : le harnais place à côté de `WAC.exe` un `wac.yml` qui éteint tous les autres, pour un cycle de quelques minutes quand un seul artefact a changé. Les contrôles d'un artefact éteint sont sautés (son JSON dit `NotRequested`) ; les artefacts lus dans les ruches demandent aussi `registry`, et l'étape 8 (`wac.yml`) est sautée. Un cycle complet reste de règle avant un commit qui touche le cœur.

Un second disque fixe de 2 Go (`create-vm.sh`) est formaté NTFS au premier cycle, étiquette `WACDATA` : il porte la `$MFT` et le journal d'un volume autre que le système, avec leurs témoins.

Le cycle enchaîne : envoi de `WAC.exe` et `raw_hive_test.exe` dans la VM, validation de l'extraction brute d'une ruche (elle doit être refusée par `reg load` tant qu'elle n'est pas réparée, puis acceptée), collecte par WAC puis conversion de cette collecte — WAC ne fait jamais les deux dans la même exécution —, puis rapatriement, dans le dossier `vmtest/results/<horodatage>/`, de :

- `run.log` — la sortie console de la collecte, qui se termine par `END, Time elapsed` quand WAC est allé au bout ;
- `raw.log` — la trace du lecteur NTFS ;
- les JSON des artefacts ;
- `exhibits/` — le manifeste de consigne, son sceau et la liste des fichiers réellement présents dans la consigne.

L'étape 7 sépare ensuite collecte et conversion (chapitre « Collecter ici, convertir ailleurs » de la documentation utilisateur) : `--collect --events --binary`, qui ne doit rien convertir ; une conversion d'une consigne dont une pièce a été retouchée, qui doit être refusée avant toute lecture, avec sa raison ; une conversion, dont les JSON passent les contrôles ci-dessous ; une seconde conversion, dont les JSON doivent être identiques à la première, octet pour octet.

Résultats dans `split/` et `split-again/`.

Autour de cette collecte seule, le cycle contrôle l'**innocuité** de la lecture : `vmtest/guest/harmlessness.ps1` relève, avant et après, les trois dates et le numéro USN de deux fichiers témoins que `--binary` prélève (un document à macro, un script aux dates falsifiées en 2001). Ils doivent être identiques. Une lecture du script par Windows (`Get-Content`) doit ensuite changer sa date d'accès : sans cela, le contrôle ne prouverait rien des dates. Les relevés sont dans `harmlessness.txt` ; la démarche est dans le *Dossier d'innocuité*. Avant cette collecte, le cycle attend 90 s après son `taskkill` : Windows écrit une modification du registre sur le disque une minute environ après l'avoir faite, et le contrôle BAM la cherche dans la ruche lue sur le disque.

3.2 Contrôler les sorties

```
python3 lib/check-json.py results/<horodatage>
```

Chaque ligne commence par une coche verte (contrôle passé), une croix rouge (défaut) ou un signe « passer » (contrôle sans objet : l'artefact est absent ou vide). Le code de sortie vaut 0 si aucun contrôle n'a échoué. Au-delà de la validité JSON et de la forme des chemins, le script confronte des sources indépendantes :

Contrôle croisé	Ce qu'il a déjà trouvé
empreinte d'un Prefetch / suffixe de son nom de fichier	65 empreintes fausses sur 270 (hexadécimal sans remplissage)
nom de l'exécutable d'un Prefetch / début du nom de fichier	un nom lu 8 octets trop tôt (« \x11 » partout)
chemins `\VOLUME{guid}` d'un Prefetch résolus	20 066 chemins sur 20 066 non résolus
couples `X` / `XUtc`, listes comprises	double décalage horaire ; `RunsUtc` des Prefetch décalés de 2 h
décalage de chaque date locale / base tz	décalage du jour de collecte appliqué à toutes les dates ; dates de changement d'heure mal lues
aucune session antérieure au démarrage	décalages de dates, heure de démarrage en retard de 3,5 s
date ShimCache / date NTFS du manifeste	une date UTC traitée comme locale (-2 h)
entrées DestList / raccourcis des listes de sauts	les 15 premières entrées de chaque liste perdues
nom de machine des événements / `OperatingSystem.json`	contrôle du décodage BinXML
identifiants d'enregistrement uniques par journal	relecture d'un chunk périmé

Contrôle croisé	Ce qu'il a déjà trouvé
montages / `reg query` et `Get-Partition`	chemins de périphérique publiés comme « \ »
précision des dates FAT et des dates lues en texte	sept chiffres de fraction sur des dates à la seconde
BAM, UserAssist, USBSTOR, Amcache / harnais, Prefetch, Windows, en-têtes PE	dates UTC lues comme locales (2 h)
noms des SID / `Get-LocalUser`, `Get-LocalGroup`	noms demandés au système, traduits (« Système »)
GUID enregistrés par le harnais (`register-test-guids.ps1`)	« Unmapped GUID » publié comme un nom
sceau / empreinte réelle du manifeste	retouche du manifeste
fichiers de la consigne / contenu du manifeste	209 pièces ajoutées après le scellement
SHA-256 de fichiers lus en brut / `Get-FileHash`	garde-fou de la lecture par lots de clusters
dates des Prefetch / dates données par Windows	0 sur 335 conformes : dates de la copie de travail
exécutables de System32 et SysWOW64 / manifeste d'un `--collect --binary`	6 446 manquants sur 8 540 (`$INDEX_ROOT` en enregistrement d'extension, liens physiques)
catalogues cités dans un verdict / pièces de la consigne	170 sur 170 absents (libellé « catalog » cherché comme « catalogue »)
verdicts Microsoft de WAC / `Get-AuthenticodeSignature`	garde-fou : un mauvais verdict laisserait un binaire hors de la consigne
copie d'un paquet du Store (intact, modifié, ajouté) / verdicts	vérification par la carte des blocs signée
deux conversions d'une même collecte	dates lues sur la copie de travail, refaite à chaque conversion
dates falsifiées d'un script / `$STANDARD_INFORMATION` et `$FILE_NAME`	garde-fou : la falsification doit rester visible
clés de récupération / en-têtes PE lus par PowerShell	garde-fou du contenu récupérable
`--binary-all` / copie et verdict de chaque exécutable	contrôle des paquets trop strict en `--binary-all`
`verify-exhibits.ps1` / collecte séparée	pièces « empreinte seule » déclarées altérées

La liste complète, avec l'histoire de chaque contrôle, est dans `vmtest/README_FR.md`.

3.3 Comparer deux passages

Une pastille verte ne dit pas si une donnée a disparu. Après tout changement, comparer la nouvelle collecte à la précédente : nombre d'entrées de chaque artefact, et valeurs perdues ou gagnées. Deux précautions :

— une collecte change d'elle-même d'un passage à l'autre : horodatages, nouveaux événements, fichiers Prefetch mis à jour, empreinte de `$MFT`. On compare donc les éléments *appariés* — par exemple un Prefetch par son empreinte de chemin et sa dernière exécution — en écartant les dates de la copie extraite ;

— une valeur « perdue » peut n'être qu'un champ vide qui n'est plus émis, ou une clé renommée : normaliser avant de conclure.

CHAPITRE 4

Les harnais de format

Chacun confronte un décodeur de WAC à une référence. Tous sont exclus de la compilation de WAC par le suffixe `_test.cpp`.

4.1 Empreintes : `sha_test`

Confronte SHA-1 et SHA-256 aux vecteurs publiés de FIPS 180-4, et aux longueurs 55 à 128 octets, qui exercent le remplissage — le seul endroit où une implémentation par ailleurs juste se trompe.

```
cd sources && g++ -std=c++17 -I. sha.cpp sha_test.cpp -o /tmp/sha_test && /tmp/sha_test
```

4.2 Lecture JSON : `json_test`

`--convert` relit ce que `--collect` a écrit : le manifeste des pièces et l'instantané de l'état en direct. Le harnais vérifie que tout ce que l'écrivain de WAC produit se relit à l'identique, que les fichiers d'une vraie collecte se relisent et se réécrivent à l'octet près, que les textes malformés sont refusés, et que 200 000 mutations aléatoires ne font jamais lire hors borne (AddressSanitizer, UBSan).

```
cd sources && g++ -std=c++17 -g -fsanitize=address,undefined -I. json_test.cpp -o /tmp/
    json_test
/tmp/json_test ../vmtest/results/<horodatage>/*.json
```

4.3 Décompression : `lznt1_test` et `xpress_test`

Vérifient les décompresseurs NTFS (LZNT1) et WOF (XPRESS Huffman) contre le *compresseur de Microsoft* : un fichier connu est compressé dans la VM par `RtlCompressBuffer`, unité par unité, et le harnais décompresse chaque unité puis la compare octet pour octet à l'original. Un décompresseur faux décode le plus souvent sans erreur : seule la comparaison exhaustive le révèle.

```
cd WAC
g++ -std=c++17 -I. lznt1.cpp lznt1_test.cpp -o /tmp/lznt1_test
g++ -std=c++17 -I. xpress.cpp xpress_test.cpp -o /tmp/xpress_test
/tmp/lznt1_test  <original> <compressé.bin> <index.idx>
/tmp/xpress_test <original> <compressé.bin> <index.idx>
```

`index.idx` porte une ligne `<n> <taille originale> <taille compressée>` par unité ; une taille compressée nulle désigne une unité que Windows a renoncé à compresser.

4.4 Artefacts par version de Windows : `artefact_catalogue_test`

Vérifie la table qui dit, pour chaque version de Windows, quels artefacts existent, où et dans quel format (`artefact_catalogue.cpp`). Cette table décide de ce qu'une collecte ne prend *pas* : une ligne fausse perdrait une pièce sans un mot. Le test contrôle que la table et `wac.yml` portent exactement les mêmes clés, que les formes de chaque artefact sont ordonnées, que les cas connus répondent juste (pas de SRUM avant Windows 8, Prefetch 23 sous Vista et 30 sous Windows 10), que le garde-fou n'écarte sous Windows 7 que les cinq artefacts que cette version n'a pas, et qu'un Windows plus récent que la table est traité comme le dernier connu, et signalé. Une clé ou une version inconnue ne retient jamais une collecte. Il contrôle aussi qu'une source n'est dite « toujours présente » que là où elle l'est : ni sur un serveur, ni pour une clé qui n'existe qu'après le branchement d'un périphérique, ni pour un métafichier du volume ou un état du système en marche. Enfin, qu'une pièce n'est décodée que si la table porte sa signature avec un décodeur : un Prefetch de version 23 est nommé et non décodé, un Prefetch de version 32, que personne n'a encore vu, n'a ni forme ni décodeur. Vu échouer : SRUM déclaré présent depuis XP ; `USBSTOR` déclaré toujours présent.

```
cd sources && g++ -std=c++17 -g -fsanitize=address,undefined -I. artefact_catalogue.cpp
    artefact_catalogue_test.cpp -o /tmp/act
/tmp/act wac.yml
/tmp/act wac.yml --dump    # la table, pour doc/artefact-table.py
```

Dans le cycle, la version que WAC lit dans la ruche `SOFTWARE` copiée est confrontée à celle que Windows donne de lui-même (numéro de build, poste de travail ou serveur).

La dernière étape du cycle écarte le fichier `hosts` le temps d'une collecte de ce seul artefact, puis le remet en place. La conversion doit donner à `hosts.json` le statut `ExpectedButMissing`, avec le build que Windows annonce et l'emplacement où le fichier aurait dû se trouver, et consigner le constat. La référence indépendante est le manifeste de la consigne : il porte les autres fichiers du dossier — le dossier a donc bien été lu — et aucun `hosts`. Aucun autre artefact ne doit porter ce statut ; dans le reste du cycle, où rien ne manque à la machine de test, il est une faute. Vu échouer sur une collecte ordinaire.

Un format sans décodeur est présenté aux décodeurs eux-mêmes. `parsers_test` réécrit la version d'un vrai Prefetch en la version suivante : elle doit être déclarée non prise en charge, le nom et l'empreinte lus, rien d'autre décodé. `file_artefacts_test` fait de même pour un index `$I` de version 3, un cache de miniatures de Vista et un DestList de jump list de version 5 ou 7 (la version 6 de Windows 11, mesurée avec `olefile`, est lue). Dans le cycle, un fichier Prefetch écrit à la main (version 32, jamais vue) est posé dans `C:\Windows\Prefetch` : `prefetchs.json` doit le déclarer sans nommer de format, et ne déclarer aucun vrai Prefetch — toutes les versions depuis celle de XP sont décodées. Chaque vrai Prefetch de la machine est ensuite lu par libscca, écrit indépendamment de WAC : version, empreinte, nombre d'exécutions, dates d'exécution à 100 ns près, numéros de série des volumes et nombre de fichiers doivent être les siens.

4.5 Sessions des navigateurs : `browser_sessions_test`

Vérifie le décodage des fichiers de session de Chromium (SNSS : `Session_*`, `Tabs_*`) et des fichiers mozLz4 de Firefox (`.jsonlz4`). Les blocs LZ4 sont produits par le module `lz4` de Python, une implémentation indépendante : chacun doit se décompresser à l'octet près. Les fichiers SNSS sont écrits par le test champ par champ, comme Chromium les pose (Pickle, champs alignés sur 32 bits) : une navigation remplacée par une plus récente, une navigation ancienne sans les champs qui suivent le titre, un onglet fermé et son instant, une commande tronquée, une longueur mensongère. Puis chaque entrée, abîmée au hasard (graine fixe), ne doit jamais faire lire hors borne (AddressSanitizer, UBSan). Vu échouer : sans l'alignement des champs, et avec une copie LZ4 qui ne se recouvre pas.

```
cd sources && g++ -std=c++17 -g -fsanitize=address,undefined -I. browser_sessions.cpp
    browser_sessions_test.cpp -o /tmp/bst
python3 -c "import lz4.block,sys; open(sys.argv[2],'wb').write(lz4.block.compress(open(sys
    .argv[1],'rb').read(), store_size=False))" a.bin a.lz4
/tmp/bst a.bin a.lz4 [<original> <bloc> ...]
```

Dans le cycle, les sessions de Chromium sont confrontées à `History` : chaque page que l'historique connaît porte l'instant d'une de ses visites, à la microseconde ; celles de Firefox, décompressées par le module `lz4` de Python, doivent donner les mêmes pages, états et instants de fermeture.

4.6 Décompression LZX : `lzx_test`

Le quatrième algorithme de WOF, LZX sur des blocs de 32 Kio, n'est produit que sur demande. Les données de test viennent de Windows lui-même : dans la VM, `compact /c /exe:lzx` compresse un fichier, et `raw_hive_test --wof` en dépose le flux compressé tel qu'il est sur le disque. Le test décompresse chaque bloc et le compare à l'original, puis soumet chaque bloc tronqué et altéré au hasard (graine fixe) au décodeur, compilé avec AddressSanitizer et UBSan : aucune lecture ni écriture hors borne, et jamais une « réussite » de taille fausse.

```
cd WAC
g++ -std=c++17 -g -fsanitize=address,undefined -I. lzx.cpp lzx_test.cpp -o /tmp/lzx_test
/tmp/lzx_test <original> <flux-wof.bin>
```

Référence : `ntoskrnl.exe`, `shell32.dll` et un fichier texte, 646 blocs sur 646 identiques, 235 707 contrôles, aucun échec. Le cycle VM vérifie en outre, à chaque passage, le SHA-256 de ces trois fichiers compressés en LZX et lus en brut contre `Get-FileHash`.

4.7 Messages d'événements : `wevt_test` et `event_messages_test`

`wevt_test` vérifie la chaîne qui transforme un événement en phrase, sur les fichiers d'un vrai fournisseur : sa DLL et son satellite localisé.

```
cd sources && g++ -std=c++17 -I. pe_resource.cpp wevt.cpp wevt_test.cpp -o /tmp/wevt_test
/tmp/wevt_test gpsvc.dll fr-FR/gpsvc.dll.mui "{aea1b4fa-97d1-45f2-a64c-4d69fffd92c9}" 1002
     4001
```

`event_messages_test` fait de même à partir de la ruche SOFTWARE ; il tourne sur la machine examinée ou dans la VM, seuls endroits où la ruche et les binaires des fournisseurs sont lisibles :

```
event_messages_test <ruche SOFTWARE> <guid> [id[:version] ...]
```

4.8 Journaux : `evtx_test`

Lit un `.evtx` hors de Windows, sous `wine` ; construit par `build-windows.sh --test`, dans `build-windows/tests/`. Il sert à éprouver le décodeur sur des journaux que la VM ne produit pas : anciens, transférés, mal fermés, abîmés — et à comparer enregistrement par enregistrement à une implémentation indépendante (`python-evtx`).

```
wine evtx_test.exe "Z:/chemin/Security.evtx"            # bilan par journal
wine evtx_test.exe "Z:/chemin/Security.evtx" 3          # + XML de 3 enregistrements
wine evtx_test.exe "Z:/chemin/Security.evtx" --dump     # un enregistrement par ligne
wine evtx_test.exe --collect "Z:/tmp/arbre" "Z:/tmp/sortie"          # chaine complete
wine evtx_test.exe --collect-memory "Z:/tmp/arbre" "Z:/tmp/sortie2"  # meme chaine, en
    memoire
cmp /tmp/sortie/events.json /tmp/sortie2/events.json                 # doit etre muet
```

4.9 Lecteur de ruches : `offline_registry_test`

WAC lit les ruches du registre avec son propre lecteur (`offline_registry.cpp`), lié dans l'exécutable : il ne dépend plus de la bibliothèque `offreg.dll` de Microsoft, que Windows fournit et qu'il empruntait donc à la machine examinée. Ce harnais confronte les deux : il charge la DLL de Microsoft depuis le chemin donné, parcourt chaque ruche avec les deux lecteurs et compare chaque clé (nom, date, classe, compteurs, maxima, taille du descripteur de sécurité) et chaque valeur (nom, type, octets), ainsi que les cas limites du contrat : tampon trop petit, sondage de taille, valeur par défaut, clé ou valeur absente. Aucun écart n'est admis.

```
offline_registry_test.exe C:\Windows\System32\offreg.dll SYSTEM SOFTWARE SAM ntuser.dat
    ...
```

À lancer sous Windows (la VM), sur des ruches rejouées : celles de `working/` d'une collecte. Seul écart voulu : une ruche dont les journaux n'ont pas été rejoués est refusée par WAC, alors que les versions récentes de la DLL de Microsoft la lisent telle quelle — des clés périmées, sans erreur ; le harnais le signale comme attendu. Référence : 18 ruches, dont dix d'une machine réelle, soit 1 273 652 clés et 2 456 139 valeurs identiques.

4.10 Conversions : `system_conversions_test`

WAC n'importe que ce qu'exige l'interrogation du système en marche (`ADVAPI32`, `KERNEL32`, `msvcrt`, `Secur32`, `WTSAPI32`). Les conversions qu'il demandait à d'autres bibliothèques sont faites par son propre code ; ce harnais les confronte aux fonctions Windows remplacées : GUID (`StringFromGUID2`), SID dans les deux sens (`ConvertSidToStringSidW`, `ConvertStringSidToSidW`), dates OLE (`VariantTimeToSystemTime`) et noms de propriétés (`PSGetNameFromPropertyKey`), dates FAT (`DosDateTimeToFileTime`, sur toutes les dates), et l'heure d'été : le décalage de WAC dans les deux sens (`SystemTimeToTzSpecificLocalTimeEx`, `TzSpecificLocalTimeToSystemTimeEx`) sur *tous* les fuseaux de Windows, avec leurs règles année par année, aux transitions (heure sautée, heure répétée, changement de règle au 1er janvier) de 1980 à 2035 ; la lecture de `SYSTEM\...\TimeZoneInformation`, dont les dates de changement d'heure sont des `TIME_FIELDS` et non des `SYSTEMTIME` ; des règles forgées au hasard ; la relecture des dates ISO 8601 du manifeste (`iso8601UtcToFiletime`, exacte au 100 ns sur 200 000 instants, et refus des dates mal formées). Il vérifie aussi le décalage horaire du suspect dans les deux sens : une date locale convertie en UTC doit l'être avec le fuseau de la machine examinée, et non avec celui de la machine qui exécute WAC. Une date impossible (mois 13, heure 25) donne une date nulle, donc non émise. La table des noms de propriétés est générée depuis le schéma d'un Windows de référence par `vmtest/guest/property-schema.cpp`.

```
system_conversions_test.exe
```

À lancer sous Windows (la VM). Référence : 9 861 570 comparaisons, aucun écart. Deux écarts voulus : WAC refuse un SID écrit que Windows tronquerait (sous-autorité de 2^{32} ou plus, seizième sous-autorité), et une date NaN, pour laquelle Windows rend une heure dénuée de sens.

4.11 Consigne : `consigne_test`

Rejoue la chaîne de production — empreintes, manifeste, copie vérifiée, rejeu des ruches — sur des ruches données en argument, et vérifie ce qu'aucune compilation ne révèle : la consigne reste identique octet pour octet pendant que la copie de travail est modifiée, et le sceau porte l'empreinte réelle du manifeste. Il vérifie enfin qu'un chemin de pièce lu dans un manifeste reste dans la consigne : un composant `..` est refusé, un nom de WinSxS qui contient `..` (`i..sermode`) est accepté.

```
cd build-windows/tests
wine consigne_test.exe "Z:/tmp/sortie" "Z:/chemin/SYSTEM" "Z:/chemin/ntuser.dat"
```

4.12 Rejeu des journaux de ruche : `hive_recover_test`

Un rejeu qui écarte une entrée, ou qui en applique une périmée, donne une ruche qui s'ouvre et se lit encore : seules manquent les clés des pages perdues, sans erreur. Le test forge une ruche et ses journaux où chaque page porte une lettre et chaque entrée en écrit une autre : la lettre restée dans une page dit quelle entrée y a été appliquée en dernier. Il vérifie

que l'entrée dont le numéro est celui de la ruche est appliquée — elle était écartée, une entrée perdue à chaque rejeu —, que les entrées antérieures et les restes d'une génération précédente ne le sont pas, qu'une entrée qui agrandit la ruche étend le fichier et l'en-tête, que l'en-tête finit propre avec sa somme de contrôle, que le journal d'annulation rend la ruche d'origine, et qu'une entrée altérée arrête la chaîne.

```
cd build-windows/tests
wine hive_recover_test.exe "Z:/tmp/travail"
```

Référence : 11 contrôles, aucun échec ; quatre échouent si la comparaison des numéros de séquence est remise à son ancienne forme. Sur des ruches réelles, le cycle VM confronte le rejeu de WAC à la récupération que Windows fait lui-même des mêmes fichiers (clés, valeurs et empreinte de l'ensemble).

4.13 Clé de hachage de la SAM : `sam_secrets_test`

Une constante fausse dans le schéma RC4 de la SAM (Windows 8.1 et antérieurs, et les comptes que rien n'a réécrits depuis) donne une clé de hachage fausse, puis des condensats NT faux pour tous les comptes, sans aucune erreur. Le test construit une valeur `F` à partir d'une clé de démarrage et d'une clé de hachage synthétiques, chiffrée et dotée de sa somme de contrôle par un calcul indépendant ; il vérifie que la clé de hachage revient, et qu'une clé de démarrage fausse, une somme de contrôle altérée, une valeur tronquée ou une révision inconnue sont refusées : aucune clé plutôt qu'une clé fausse.

```
cd build-windows/tests
wine sam_secrets_test.exe
```

Référence : 5 contrôles, aucun échec ; le premier échoue avec l'ancienne constante (treize « Q » au lieu de douze). Sur une SAM réelle, le cycle VM confronte le condensat d'un compte témoin au MD4 de son mot de passe, sous Windows 11 (AES) et sous Windows 8.1 (RC4).

4.14 Scripts de vérification : `verify_exhibits_test.sh`

Lance `verify-exhibits.sh` et `verifier-consigne.sh`, les scripts Linux livrés avec `WAC.exe`, sur des collectes forgées dont le verdict est connu : une collecte intacte (un chemin avec une espace, une empreinte en minuscules, un binaire à empreinte seule) est acceptée ; une pièce altérée, absente ou sans empreinte, un manifeste modifié ou illisible sont refusés. Un manifeste qui désigne un fichier de la machine d'investigation — par `..`, par un chemin absolu ou par un lien symbolique — est refusé sans que le fichier soit lu. Les deux scripts passent chaque cas, avec le même code de retour et les mêmes compteurs attendus : l'un ne peut pas dériver de l'autre sans que cela se voie. Linux seulement, aucune donnée nécessaire.

```
bash sources/verify_exhibits_test.sh
```

Référence : 23 contrôles, aucun échec. Les scripts `.ps1` sont lancés par le cycle VM sur la collecte séparée.

4.15 Lecture brute : `raw_hive_test`

Extrait un fichier en lisant directement le volume NTFS. À lancer sous Windows, en administrateur ; le cycle VM l'utilise pour valider l'extraction d'une ruche.

```
raw_hive_test.exe [volume] [chemin] [sortie] [--fix] [--list] [--attrs]
```

`--fix` rend la ruche extraite chargeable, `--list` liste un répertoire, `--attrs` affiche les attributs `$MFT` du fichier sans les interpréter.

4.16 Enregistrements NTFS : `ntfs_fixup_test`

NTFS écrit un enregistrement de la `$MFT` ou un bloc d'index secteur par secteur, et termine chaque tranche de 512 octets par un numéro de séquence. WAC vérifie ce numéro avant de restituer les vrais octets : un enregistrement déchiré par une écriture interrompue est refusé. Le harnais construit des enregistrements dont le résultat attendu est connu (sains, déchirés dans chaque tranche, tableaux malformés), puis confronte le verdict de WAC à une vérification indépendante sur 20 000 enregistrements aléatoires placés contre une page interdite. Aucune donnée n'est nécessaire.

```
cd build-windows/tests && wine ntfs_fixup_test.exe
```

Référence : 44 610 contrôles, aucun échec. Sur un vrai volume, le cycle de la VM le confirme : `raw.log` ne doit contenir aucun « torn record ».

4.17 `$MFT` et journal des modifications : `mft_test`, `usn_journal_test`

Un défaut de décalage dans la lecture de la `$MFT` ou du journal des modifications donne des dates, des noms et des références valides et faux : aucun contrôle de syntaxe ne le verrait. Les deux harnais comparent donc WAC à une référence indépendante.

`mft_fixtures.sh` formate un volume NTFS dans un fichier avec `mkntfs`, y dépose des fichiers (résidents, non résidents, nom non ASCII) avec `ntfscp`, sans montage ni privilège, extrait sa `$MFT` avec `ntfscat`, puis fait décrire chaque enregistrement par `ntfsinfo` — ntfs-3g, une autre implémentation de NTFS. `mft_test` confronte, enregistrement par enregistrement, le nom, le parent, la taille, les huit dates et les flux nommés lus par WAC à cette description. `usn_journal_test` construit des enregistrements de versions 2 et 3 avec des structures dont la disposition vient du compilateur (`USN_RECORD` de MinGW ; la version 3, que MinGW ne déclare pas, d'après la documentation de Microsoft, ses décalages vérifiés à la compilation), les assemble en journal tel que NTFS le laisse (pages, zéros, enregistrement de plage, zone creuse de tête), vérifie les numéros, les motifs et les chemins, puis décode chaque enregistrement abîmé au hasard contre une page interdite.

```
bash sources/mft_fixtures.sh /tmp/mft          # ntfs-3g : mkntfs, ntfscp, ntfsinfo
cd build-windows/tests
wine mft_test.exe "Z:/tmp/mft"
wine usn_journal_test.exe "Z:/tmp/mft"
```

Référence : 278 contrôles pour `mft_test`, 27 pour `usn_journal_test`, aucun échec.

4.18 Lecture brute sur un vrai volume : `raw_volume_test.sh`

L'index d'un répertoire désigne un fichier par son numéro d'enregistrement *et* un numéro de séquence, que NTFS incrémente chaque fois qu'il libère l'enregistrement. Sur le système en marche, un fichier peut être supprimé entre la lecture de son répertoire et celle de son contenu, et son enregistrement donné à un autre fichier : WAC le lisait quand même, et scellait le contenu de cet autre fichier sous le nom du premier (un `SRU*.log` contenant les blocs `hbin` d'une ruche, relevé dans la VM de test). Le script sert à Wine le volume de `mft_fixtures.sh` comme lecteur `T:`, dans un préfixe Wine à lui, et le fait lire par `raw_hive_test` comme WAC lit un disque : un fichier dont les octets doivent être ceux que lit `ntfscat`, puis le même fichier une fois son enregistrement doté d'une autre séquence que son entrée d'index — refusé, sans copie laissée.

```
bash sources/mft_fixtures.sh /tmp/mft
bash sources/raw_volume_test.sh /tmp/mft build-windows/raw_hive_test.exe
```

Référence : 3 contrôles, aucun échec ; le lecteur d'avant le correctif en échoue 2.

4.19 Courriels d'Outlook : `pst_test`

Un PST lu de travers — un bloc déchiffré avec la mauvaise table, un élément du tas coupé d'un octet, un bloc compressé décompressé trop court — donne encore un JSON valide. `pst_fixtures.sh` télécharge, épinglés par leur SHA-256, les fichiers de test de java-libpst (Apache 2.0 : deux PST Unicode et un OST d'Outlook 2013, pages de 4 Kio et blocs compressés) et un PST du corpus EDRM Enron v2 (CC-BY 3.0 US), et fait lire chaque message par pypff, le lecteur de libpff écrit indépendamment de WAC. `pst_test` compare, message par message : objet, expéditeur, dates de remise et d'envoi, nom, taille et SHA-256 de chaque pièce jointe, SHA-256 du corps.

```
PYTHON=vmtest/tools/pff-venv/bin/python bash sources/pst_fixtures.sh /tmp/pstfx
cd build-windows/tests && wine pst_test.exe Z:/tmp/pstfx
```

Référence : 1 800 contrôles, aucun échec ; les 256 messages que lit pypff identiques, et les 4 éléments du PST Enron que pypff ne lit pas lus par WAC. Le déchiffrement retiré, 253 échecs.

4.20 Index d'Internet Explorer : `ie_index_dat_test`

Vérifie le décodeur des `index.dat` d'Internet Explorer 5 à 9 (Windows 7 et antérieurs) sur quatre vrais fichiers des données de test publiques du projet plaso — historique, cache, historique journalier, et un cache de plus de mille enregistrements avec des redirections, des entrées abandonnées (`LEAK`) et des entrées récupérées. La référence est `pymsiecf`, le lecteur de libmsiecf, écrit indépendamment de WAC : chaque enregistrement qu'il donne doit revenir avec les mêmes champs (URL, fichier, répertoire de cache, temps, expiration, dernière

vérification, hits, taille des données), et aucun de plus. Ce test a pris un défaut réel : un enregistrement récupéré, en partie écrasé par un plus récent, rendait comme siennes les données de l'autre. Puis chaque fichier est décodé tronqué et altéré au hasard, contre une page interdite des deux côtés ; enfin la conversion (conteneur, compte et page, fichier en cache, en-têtes, dates) est vérifiée.

```
pip install libmsiecf-python              # pymsiecf, pour la référence
PYTHON=python3 sources/ie_index_fixtures.sh /tmp/iefx
cd build-windows/tests && wine ie_index_dat_test.exe Z:/tmp/iefx
```

Les fichiers ne sont pas versionnés : le script les télécharge et les refuse si leur SHA-256 diffère. Référence : 22 contrôles, aucun échec (1 110 enregistrements identiques à ceux de `pymsiecf`).

4.21 Chronologie Windows : `activities_cache_test`

Vérifie la lecture de `ActivitiesCache.db` sur une vraie base de Windows 10 des données de test publiques du projet plaso (112 activités : applications, pages, temps au premier plan). La référence est le `sqlite3` de Python, chaque colonne convertie par les règles de `activities_cache.h` réécrites dans le script, indépendamment de WAC : temps Unix en UTC, GUID dans l'ordre d'octets de .NET, charges utiles JSON analysées, valeurs nulles omises. Chaque ligne doit revenir avec les mêmes colonnes et valeurs, et aucune de plus ; les formes locales des dates ne sont pas comparées (elles dépendent du fuseau de la machine examinée). Puis une copie tronquée est lue sans plantage. Réintroduire un décalage d'une heure dans la conversion des dates fait échouer 113 contrôles.

```
sources/activities_cache_fixtures.sh /tmp/acfx
cd build-windows/tests && wine activities_cache_test.exe Z:/tmp/acfx
```

La base n'est pas versionnée : le script la télécharge et la refuse si son SHA-256 diffère. Référence : 118 contrôles, aucun échec.

4.22 Petits artefacts fichiers : `file_artefacts_test`

Vérifie les décodeurs de l'historique PowerShell, de `setupapi.dev.log`, des `$I` de la corbeille, de `hosts`, de `Report.wer` et du MPLog de Defender sur des entrées écrites d'après la description de chaque format, avec le résultat attendu : une commande sur deux lignes réunie, un titre de section séparé en opération et sujet, le 31 février refusé comme date, un `$I` de version 1 et 2 lu, un `$I` tronqué ou de version inconnue refusé, une longueur de chemin mensongère bornée par le fichier, une signature de problème ordonnée par son indice quel que soit l'ordre du fichier, une date sans `Z` non prise pour de l'UTC. Pour `$Secure:$SDS` : un flux construit bloc par bloc, où le bloc miroir est sauté même quand son entrée paraît cohérente, un `SecurityId` répété gardé une fois, une longueur qui sort du bloc refusée, un condensat abîmé signalé (`HashMismatch`), le propriétaire et l'ACE décodés, chaque lecture contre une page interdite des deux côtés. Pour `$Boot` : géométrie et numéro de série lus, un nombre de secteurs par cluster au-delà de 0x80 pris comme puissance de

deux, un secteur sans 0x55AA, d'un autre système de fichiers, de 100 octets ou tronqué refusé ; pour `$Max` : les quatre champs, un flux trop court refusé. Pour une clé maîtresse DPAPI : en-tête lu, des tailles de blocs au-delà du fichier ou qui débordent en s'additionnant refusées, un en-tête tronqué refusé ; `Preferred` rendu sous la forme du nom des fichiers de clés. Pour `UserShellFolders` : `%USERPROFILE%` remplacé par le profil quelle que soit sa casse, un chemin sur un autre disque gardé, un partage ou une autre variable d'utilisateur non suivis. Pour la lecture SQLite : une base en mode WAL dont l'écrivain est encore ouvert, comme un navigateur en marche, lue sur sa copie privée avec les lignes de son journal (texte UTF-8, entier, NULL, BLOB), la base et le journal inchangés, la copie supprimée, une requête d'un autre schéma et un fichier qui n'est pas une base refusés. Pour l'historique Chromium : une base `History` construite avec les tables de Chromium, une visite saisie puis un lien qui la cite, un téléchargement redirigé dont la chaîne est rendue dans son ordre, un terme cherché ; nature et qualificatifs des transitions, temps de Chromium convertis. Pour Firefox : `places.sqlite` et `formhistory.sqlite` construits avec les tables de Firefox, une visite de téléchargement qui cite sa visite d'origine, la destination d'un téléchargement décodée de son URI `file:///` (UTF-8, espace), sa fin en millisecondes, une saisie de la barre d'adresse et une recherche de la barre de recherche. La VM n'a pas Firefox : ce test est sa seule vérification. Identifiants enregistrés et cookies de Chromium et de Firefox : site, compte, dates (expiration des cookies Firefox en secondes), et aucun mot de passe ni aucune valeur de cookie dans le JSON. Aucune donnée nécessaire.

```
cd build-windows/tests && wine file_artefacts_test.exe
```

Référence : 174 contrôles, aucun échec (dont les valeurs du lot 2 du registre : TrustRecords, règle de pare-feu, blob de certificat ; et les caches des miniatures : une en-tête d'entrée réelle de la VM et la somme que Windows lui a écrite, un cache construit, des longueurs forgées ; le courrier de Thunderbird : une boîte mbox écrite ici, mots encodés, quoted-printable, base64, nom RFC 2231, multipart imbriqué ou sans frontière, date avec décalage). Dans le cycle VM, `place-file-artefacts.ps1` dépose des témoins de contenu connu (un historique, un fichier mis à la corbeille, une entrée `hosts`, un script de démarrage) et `check-json.py` les retrouve dans les JSON.

4.23 Signatures : `authenticode_test`

Vérifie les catalogues d'un dossier puis rend un verdict par fichier ; ses verdicts se comparent à ceux de `Get-AuthenticodeSignature` sur les mêmes fichiers, dans la VM. Le seul écart qui compte : un fichier accepté par WAC et refusé par Windows.

```
authenticode_test --catalogs <dossier>
authenticode_test --catalogs <dossier> --files <liste>
```

4.24 Configuration : `yaml_light_test`, `config_test`

`yaml_light_test` (natif, libyaml compris, sous AddressSanitizer et UBSan) : lectures exactes (sections imbriquées, guillemets, commentaires, marque d'ordre des octets, fichier vide),

refus avec la bonne ligne (clé en double, liste, ancre, alias, étiquette, second document, tabulation, guillemet non fermé, imbrication trop profonde), puis 3 000 textes abîmés au hasard ; sept mutations toutes détectées. `config_test` (sous wine) : chaque réglage appliqué, chaque sorte d'erreur refusée avec sa ligne, et le fichier de référence de `--write-config` relu tel qu'il se décrit ; trois mutations détectées. Dans la VM, une collecte pilotée par un `wac.yml`, sans aucune option, est confrontée au fichier consigné dans `investigation.json` (`check-json.py --configuration`).

4.25 Chaînes tierces : `third_party_chain_test`

Chaque règle de la vérification des chaînes tierces sur des chaînes fabriquées par openssl pour l'enfreindre (`third_party_chain_fixtures.sh`) : chaîne saine acceptée ; racine absente, exclue de la signature de code, désapprouvée ; clé d'autorité désapprouvée par Microsoft ; autorité révoquée selon la CCADB ; émetteur sans `cA` ; signataire réservé au web ; autorité du bon nom mais d'une autre clé ; autorité ECDSA (non vérifiée). Puis l'heure à laquelle une chaîne est jugée, avec un vrai jeton RFC 3161 fait par `openssl ts` et l'heure de collecte portée en 2040 : signataire valide de 2020 à 2035 accepté sans horodatage aujourd'hui, refusé en 2040, accepté en 2040 s'il est horodaté aujourd'hui ; jeton sur une autre signature, ou altéré, refusé ; autorité d'horodatage sous une racine non admise pour cet usage, refusée ; racine désapprouvée après 2030 acceptée pour une signature horodatée avant, refusée sinon. Enfin la révocation, avec de vraies CRL faites par `openssl ca` : chaîne saine acceptée avec les CRL de l'autorité et de la racine ; sans elles, avec celle de l'autorité absente ou forgée (bon nom d'émetteur, autre clé), refusée comme invérifiable ; CRL trouvée par la CCADB quand l'adresse du certificat manque, celle du certificat prévalant ; signataire révoqué pour compromission refusé même horodaté avant ; révoqué comme remplacé après l'horodatage, accepté s'il est horodaté, refusé sinon. Chaque règle neutralisée tour à tour fait échouer le test (vingt-deux mutations). Les horodatages des installeurs réels concordent à la seconde avec une lecture indépendante (`openssl ts` pour les jetons RFC 3161, DER pour les contre-signatures). Les signatures réelles se recoupent par `authenticode_test --trust`, face à `openssl verify` avec le `roots.pem` du jeu (installeurs de Git, Notepad++, PuTTY, Python, VLC ; jeu complet et jeu privé des racines Sectigo) ; dans la VM, aucune chaîne jugée de confiance par WAC ne l'est pas par `Get-AuthenticodeSignature`.

```
./third_party_chain_fixtures.sh /tmp/chain
/tmp/third_party_chain_test /tmp/chain
```

4.26 Pilotes vulnérables : `version_info_test`, `vulnerable_drivers_test`

`version_info_test` confronte la lecture de la ressource de version (nom d'origine, nom interne, produit, description, version) à celle de Windows (`VersionInfo` de PowerShell dans la VM) sur 58 binaires : versions et textes identiques, hors textes qu'un binaire localisé tient de son satellite `.mui` — que la liste de blocage ne lit pas —, puis ressources tronquées et abîmées (11 852 contrôles). `vulnerable_drivers_test` prend chaque règle

seule : empreinte Authenticode ou du fichier, autorité refusée dans la chaîne, restreinte au signataire, au fabricant WHQL, au nom d'origine, au produit, aux bornes de version ; binaire sans ressource de version (la règle de fichier ne s'applique pas, comme chez Windows) ou non lu (elle s'applique) — neuf mutations toutes détectées. Sur des échantillons réels de LOLDrivers, l'empreinte Authenticode de WAC est leur « Authentihash » (4 sur 4), et deux pilotes vulnérables dont la chaîne tient sont refusés ; dans la VM, un pilote vulnérable déposé (jamais chargé) doit être reconnu.

```
g++ -std=c++17 -g -fsanitize=address,undefined -I. pe_resource.cpp \
    version_info.cpp version_info_test.cpp -o /tmp/version_info_test
/tmp/version_info_test <reference> <dossier>
```

4.27 Listes de confiance : `inflate_test`, `cab_test`, `zip_test`, `csv_reader_test`, `trust_list_test`

`--update-trust` décompresse et lit ce que Microsoft publie ; une erreur donnerait une sortie valide et fausse. Chaque maillon a donc sa référence indépendante, et se construit sous Linux avec AddressSanitizer et UBSan :

— `inflate_test` : DEFLATE, confronté à zlib sur 49 flux, puis flux tronqués et abîmés au hasard (graine fixe) ;

— `cab_test` : cabinets MSZIP, extraction identique octet pour octet à celle de `cabextract` sur les vrais `authrootstl.cab` et `disallowedcertstl.cab` (1 134 contrôles) ;

— `zip_test` : archives ZIP, extraction identique à celle de `zipfile` (Python) sur la vraie liste de pilotes vulnérables de Microsoft et deux archives synthétiques (stockée, compressée, commentaire, dossiers, fichier vide) ; une archive abîmée ne rend jamais un contenu altéré, refusé par son CRC-32 (1 717 contrôles) ;

— `csv_reader_test` : le rapport CSV de la CCADB, lu champ pour champ comme par le module `csv` de Python (10 284 enregistrements, empreinte SHA-256 de chaque champ), puis guillemets doublés, sauts de ligne dans un champ, CRLF, guillemet non fermé, enregistrement incomplet ;

— `trust_list_test` : les comptes de `authroot.stl` (racines pour la signature de code, pour l'horodatage, et pour la signature de code non désapprouvées), calculés à part par une lecture DER en Python ; les certificats que bloque Chromium (`net/data/ssl/blocklist`), reconnus par `disallowedcert.stl` ; une liste modifiée d'un octet, refusée ; les deux listes lues par huit fils à la fois, cache des chaînes vide, comme les fils d'analyse de `--collect --binary`. Construit une seconde fois avec `-fsanitize=thread` (lancé par `setarch -R`, que ThreadSanitizer exige sur les noyaux récents), il signalait onze courses de données sur ce cache avant son verrou.

```
g++ -std=c++17 -g -fsanitize=address,undefined -I. authenticode.cpp rsa.cpp \
    sha.cpp quickdigest5.cpp trust_list_test.cpp -o /tmp/trust_list_test
/tmp/trust_list_test authroot.stl 313 335 145 disallowedcert.stl blocklist 36
```

Référence (listes du 25 août 2026 et du 5 septembre 2025) : 80 contrôles, aucun échec. C'est ce test qui a établi ce que contient `disallowedcert.stl`, que Microsoft ne documente pas : ses entrées de 16 octets sont le MD5 de la **clé publique** (et non de la partie signée, comme le nom de la propriété le laisse croire), celles de 48 octets le SHA-384 de la partie signée.

Le résultat de bout en bout se contrôle dans la VM : les 562 racines écrites par `--update-trust` sont identiques, octet pour octet, à celles que produit `certutil -generateSSTFromWU`. Les listes de pilotes vulnérables se recoupent par une lecture Python du XML et du JSON téléchargés : mêmes 1 086 empreintes Microsoft, 217 signataires refusés, 8 151 empreintes LOLDrivers. Les CRL se contrôlent par openssl : chacune se lit, et son émetteur est l'autorité que lui associe la CCADB (387 sur 387).

CHAPITRE 5

Les harnais de robustesse

5.1 Le principe

Un analyseur lit à des positions que le fichier lui-même déclare. Sur un fichier tronqué ou forgé, une lecture au-delà du tampon ne plante pas d'elle-même : elle lit la mémoire voisine et la publie, dans un JSON valide. Les harnais la font planter : chaque entrée est copiée contre une page mémoire interdite (`PAGE_NOACCESS`), une fois *finissant* contre elle (lectures au-delà de la fin), une fois *commençant* juste après elle (lectures en deçà du début, par exemple à une position négative tirée de la donnée). Un seul octet lu hors de l'entrée provoque une violation d'accès.

Pour chaque fichier donné, le harnais analyse :

— le fichier entier ;

— ses troncatures (toutes les longueurs pour les petits fichiers) ;

— des copies dont des octets ont été remplacés au hasard, avec une graine fixe : un échec se reproduit à l'identique.

5.2 Raccourcis : `lnk_test`

```
cd build-windows/tests
wine lnk_test.exe a.lnk b.lnk ...
```

Sources de fichiers : le dossier `Recent` et le menu Démarrer de la VM, ou ceux d'une collecte (`Users\<nom>\AppData\Roaming\Microsoft\Windows\Recent`). Les noms accentués ou non latins passent mal dans la ligne de commande de `wine` : copier les fichiers sous des noms numérotés.

5.3 Listes de sauts et Prefetch : `parsers_test`

```
parsers_test.exe jumplist-auto   <fichier.automaticDestinations-ms> ...
parsers_test.exe jumplist-custom <fichier.customDestinations-ms> ...
parsers_test.exe prefetch        <fichier.pf> ...
parsers_test.exe hive            <ruche> ...
```

Le type `hive` éprouve le lecteur de ruches : chaque entrée est ouverte depuis la mémoire, contre la page interdite, et toute l'arborescence est parcourue (sous-clés, valeurs, `ORQuery`

`InfoKey`). Une sous-clé qui pointerait vers l'une de ses ancêtres est refusée par le lecteur : sans cela, un parcours récursif ne finirait jamais.

Les listes de sauts se testent sous `wine`, comme `lnk_test`. Les Prefetch se testent **dans la VM** : ils sont compressés (XPRESS Huffman), et seule la bibliothèque `ntdll` de Windows les décompresse. Le harnais décompresse chaque fichier avant de l'éprouver : c'est l'analyseur de WAC qu'il vise, pas le décompresseur de Windows.

Pour les 307 Prefetch d'une machine réelle (plus d'un million d'entrées), il faut compter environ deux heures dans la VM : lancer le test en tâche détachée, par lots, et suivre son journal. Exemple de script PowerShell, déposé dans la VM avec `qga.py write` :

```
$files = Get-ChildItem C:\wactest\pt\pf\*.pf | ForEach-Object { $_.FullName }
$log = 'C:\wactest\pt\pf_results.txt'
for ($i = 0; $i -lt $files.Count; $i += 20) {
  $batch = $files[$i..([Math]::Min($i + 19, $files.Count - 1))]
  Add-Content $log (& C:\wactest\pt\parsers_test.exe prefetch @batch 2>&1)
  Add-Content $log ("batch " + $i + " exit=" + $LASTEXITCODE)
}
Add-Content $log 'DONE'
```

5.4 Référence

Résultats obtenus sur une machine réelle, chaque entrée analysée des deux côtés de la page interdite :

Fichiers	Entrées	Où
220 raccourcis	546 214	`wine`
48 listes de sauts automatiques	175 585	`wine`
23 listes de sauts personnalisées	79 036	`wine`
307 Prefetch	1 246 491	VM, environ 2 h
4 ruches du registre	16 207	`wine`

CHAPITRE 6

Recouper sur une collecte réelle

Une VM fraîchement installée ne produit ni fragmentation, ni listes de sauts anciennes, ni noms de fichiers exotiques. Plusieurs défauts ne se sont révélés que sur une collecte réelle, confrontée à une source indépendante :

— **listes de sauts / `olefile`** : les flux et le DestList, relus par un lecteur OLE indépendant, doivent donner exactement les mêmes entrées (numéros, dates, chemins). C'est ce qui a montré que les flux s'appellent « 1 » à « f » et non « 01 » à « 0f » ;

— **raccourci / DestList** : dans une liste de sauts, la cible d'un raccourci et le chemin de son entrée DestList sont deux enregistrements du même fichier. Leur désaccord a révélé une page de code fausse (« d\x92orientation ») et un chemin Unicode ignoré ;

— **événements / `python-evtx`** : comparaison enregistrement par enregistrement (`evtx_test --dump`) ;

— **signatures / `Get-AuthenticodeSignature`** (`authenticode_test`) ;

— **ancienne collecte / nouvelle** : décoder les mêmes fichiers avec le code courant et comparer, après normalisation, à la sortie d'une version précédente.

> **Attention**
> Une collecte réelle se traite en local. Si une ruche doit être confrontée au lecteur de Microsoft (`offline_registry_test`), elle peut être copiée dans la VM de test, puis effacée aussitôt le test fini. Aucune copie ne reste dans le dossier de travail temporaire, et rien n'en entre dans le dépôt.

CHAPITRE 7

Lire un échec

7.1 Codes de sortie

Code	Signification
0	tout est passé
1	échec d'un contrôle, ou fichier d'entrée illisible
2	mauvais usage (arguments)
3	`parsers_test` : une analyse ne s'est pas terminée en 60 s (chien de garde)
`0xC0000005` (-1073741819)	violation d'accès : lecture hors du tampon

7.2 Localiser une lecture hors tampon

Sous `wine`, une faute affiche la pile d'appels, avec des adresses relatives au programme (`lnk_test+0x485bb`). Sous Windows, `parsers_test` affiche lui-même l'entrée en cause et l'adresse fautive :

```
FAILED  exception 0xC0000005 at exe+0x78b70, address read 0x159deb4200c:
        C:\wactest\pt\pf\CMD.EXE-0BD30981.pf truncated at 192 (guard after)
```

L'adresse se traduit en fonction avec la table des symboles du programme (adresse de base `0x140000000`) :

```
x86_64-w64-mingw32-nm -C parsers_test.exe | grep -i " t " | sort > syms.txt
# chercher le symbole dont l'adresse est juste inférieure à 0x140000000 + 0x78b70
x86_64-w64-mingw32-objdump -d --start-address=0x140078b40 \
    --stop-address=0x140078b80 parsers_test.exe
```

L'instruction désassemblée donne le décalage lu (ici `0xcc(%r14)`) ; il se rapproche alors, dans le code source, du champ lu à cette position. Rejouer ensuite l'entrée exacte (la troncature ou la corruption indiquée) pour vérifier la correction, puis toute la série.

7.3 Une boucle sans fin

Si `parsers_test` s’arrête sur « no end after 60 s », l’entrée nommée fait tourner un analyseur sans fin — typiquement une chaîne de secteurs cyclique dans un conteneur OLE. C’est un défaut au même titre qu’une lecture hors tampon : un fichier forgé suffirait à bloquer une collecte.